ICPC 2026

PROBLEM	READ	SOLVED	WRITTEN	OPENED	COMMENT
A					
B					
C					
D					
E					
F					
G					
H					
I					
J					
K					
L					
M					
N					

Time	Andrey	Taisia	Timofey
0:30			
1:00			
1:30			
2:00			
2:30			
3:00			
3:30			
4:00			
4:30			
5:00			

Contents

1 Combinatorics

1.1 Gray Code

```
inline ll g(ll n) {
    return n ^ (n >> 1);
}
```

2 Convolutions

2.1 FFT MOD

```
const ll ROOT = 31;
const ll REV_ROOT = rev(ROOT);
const int ROOT_PW = (1ll << 23);

int bit_rev(int num, int lg_n) {
    int res = 0;
    for (int i = 0; i < lg_n; ++i)
        if (num & (1 << i))
            res |= 1 << (lg_n - 1 - i);
    return res;
}

void fft_inpl(vector<ll>& a, bool invert) {
    int n = a.size();
    int lg_n = 0;
    while ((1 << lg_n) < n)
        ++lg_n;

    for (int i = 0; i < n; ++i) {
        if (i < bit_rev(i, lg_n)) {
            swap(a[i], a[bit_rev(i, lg_n)]);
        }
    }

    for (int len = 2; len <= n; len <<= 1) {
        ll w = invert ? REV_ROOT : ROOT;
        for (int i = len; i < ROOT_PW; i <<= 1)
            w = (w * w) % MOD;
        for (int i = 0; i < n; i += len) {
            ll wn = 1;
            for (int j = 0; j < len / 2; ++j) {
                ll u = a[i + j], v = (a[i + j + len / 2] * wn) % MOD;
                a[i + j] = u + v < MOD ? u + v : u + v - MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
                wn *= w;
                wn %= MOD;
            }
        }
    }
    if (invert) {
        int nrev = rev(n);
        for (int i = 0; i < n; ++i) {
            a[i] *= nrev;
            a[i] %= MOD;
        }
    }
}

vector<ll> vect_mult(const vector<ll>& a, const vector<ll>& b) {
    ll sz = a.size() + b.size() - 1;
    ll n = 1;
    while (n < sz) {
        n <<= 1;
    }

    vector<ll> fa(n, 0), fb(n, 0);
    for (int i = 0; i < n; ++i) {
        if (i < a.size()) {
            fa[i] = a[i];
        }
        if (i < b.size()) {
            fb[i] = b[i];
        }
    }

    fft_inpl(fa, false);
    fft_inpl(fb, false);
    for (int i = 0; i < n; ++i) {
        fa[i] *= fb[i];
        fa[i] %= MOD;
    }
    fft_inpl(fa, true);
```

```
    vector<ll> mult(sz);
    for (int i = 0; i < sz; ++i) {
        mult[i] = fa[i];
    }
    return mult;
}
```

2.2 FFT

```
int bit_rev(int num, int lg_n) {
    int res = 0;
    for (int i = 0; i < lg_n; ++i)
        if (num & (1 << i))
            res |= 1 << (lg_n - 1 - i);
    return res;
}

// FFT
void fft_inpl(vector<comp>& a, bool invert) {
    int n = a.size();
    int lg_n = 0;
    while ((1 << lg_n) < n)
        ++lg_n;

    for (int i = 0; i < n; ++i) {
        if (i < bit_rev(i, lg_n)) {
            swap(a[i], a[bit_rev(i, lg_n)]);
        }
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        comp w(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            comp wn(1);
            for (int j = 0; j < len / 2; ++j) {
                comp u = a[i + j], v = a[i + j + len / 2] * wn;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                wn *= w;
            }
        }
    }
    if (invert) {
        for (int i = 0; i < n; ++i) {
            a[i] /= n;
        }
    }
}

vector<ll> vect_mult(const vector<ll>& a, const vector<ll>& b) {
    ll sz = a.size() + b.size() - 1;
    ll n = 1;
    while (n < sz) {
        n <<= 1;
    }

    vector<comp> fa(n, 0), fb(n, 0);
    for (int i = 0; i < n; ++i) {
        if (i < a.size()) {
            fa[i] = comp(a[i], 0);
        }
        if (i < b.size()) {
            fb[i] = comp(b[i], 0);
        }
    }

    fft_inpl(fa, false);
    fft_inpl(fb, false);
    for (int i = 0; i < n; ++i) {
        fa[i] *= fb[i];
    }
    fft_inpl(fa, true);

    vector<ll> mult(sz);
    for (int i = 0; i < sz; ++i) {
        mult[i] = round(fa[i].real());
    }
    return mult;
}
```

2.3 FHT

```
inline ll soft_mod(ll a) {
    return a >= MOD ? a - MOD : a;
}

void FHT(vector<ll>& a) {
```

```
    ll sz = a.size();
    for (int mid = 1; mid < sz; mid <<= 1) {
        for (int i = 0; i < sz; i += mid << 1) {
            for (int j = 0; j < mid; ++j) {
                ll x = a[i + j];
                ll y = a[i + j + mid];
                a[i + j] = soft_mod(x + y);
                a[i + j + mid] = soft_mod(x - y + MOD);
            }
        }
    }
}

vector<ll> xor_mult(vector<ll> a, vector<ll> b) {
    FHT(a);
    FHT(b);
    ll n = a.size();
    vector<ll> c(n);

    for (int i = 0; i < n; ++i) {
        c[i] = (a[i] * b[i]) % MOD;
    }

    FHT(c);
    ll revn = bin_pow(n, MOD - 2);
    for (int i = 0; i < n; ++i) {
        c[i] *= revn;
        c[i] %= MOD;
    }
    return c;
}
```

2.4 SOS DP

```
void sos(vector<ll>& dp) {
    ll n = dp.size();
    ll k = 0;
    while ((1ll << k) < n) {
        ++k;
    }

    for (int j = 0; j < k; ++j) {
        for (int i = 0; i < n; ++i) {
            if (i & (1ll << j)) {
                dp[i] += dp[i ^ (1ll << j)];
            }
        }
    }
}

void revsos(vector<ll>& dp) {
    ll n = dp.size();

    ll k = 0;
    while ((1ll << k) < n) {
        ++k;
    }

    for (int j = k - 1; j >= 0; --j) {
        for (int i = 0; i < n; ++i) {
            if (i & (1ll << j)) {
                dp[i] -= dp[i ^ (1ll << j)];
            }
        }
    }
}
```

3 Data Structures

3.1 Centroid Decomposition

```
struct CentroidDecomp {
    ll n;
    vector<ll> parent;

    void calc_sizes(ll v, vector<char>& used, vector<vector<ll>>& g,
    vector<ll>& s) {
        s[v] = 1;
        used[v] = 1;

        for (ll u : g[v]) {
            if (!used[u]) {
                calc_sizes(u, used, g, s);
                s[v] += s[u];
            }
        }
    }
```

```
ll find_centroid(ll v, vector<char>& used, vector<vector<ll>>& g,
vector<ll>& s, ll sz) {
    used[v] = 1;

    for (ll u : g[v]) {
        if (!used[u]) {
            if (s[u] > sz / 2) {
                return find_centroid(u, used, g, s, sz);
            }
        }
    }

    return v;
}

void set_parent(ll v, vector<char>& used, vector<vector<ll>>& g, ll
cent) {
    if (v != cent)
        parent[v] = cent;
    used[v] = 1;

    for (ll u : g[v]) {
        if (!used[u]) {
            set_parent(u, used, g, cent);
        }
    }
}

CentroidDecomp(ll N, vector<vector<ll>>& g) {
    n = N;
    parent.assign(n, -1);

    vector<char> cused(n, 0);
    vector<ll> s(n, 0);
    ll cnt_used = 0;

    while (cnt_used < n) {
        vector<char> sused = cused, used = cused, pused = cused;

        for (int i = 0; i < n; ++i) {
            if (!sused[i]) {
                calc_sizes(i, sused, g, s);
                ll cent = find_centroid(i, used, g, s, s[i]);
                cused[cent] = 1;
                ++cnt_used;
                set_parent(i, pused, g, cent);
            }
        }
    }
};
```

3.2 DSU

```
struct DSU {
    ll n;
    vector<ll> prev, size;

    DSU(ll N) : n(N) {
        prev.assign(n, -1);
        size.assign(n, 1);
    }

    ll root(ll a) {
        ll aa = a;
        while (prev[a] != -1)
            a = prev[a];

        while (prev[aa] != -1) {
            ll last = prev[aa];
            prev[aa] = a;
            aa = last;
        }
        return a;
    }

    bool connected(ll a, ll b) {
        return root(a) == root(b);
    }

    bool unite(ll a, ll b) {
        a = root(a);
        b = root(b);
        if (a == b)
            return false;
        if (size[a] < size[b])
```

```
            swap(a, b);
        prev[b] = a;
        size[a] += size[b];
        return true;
    }
};
```

3.3 Fenwick 2D

```
template <class T>
struct Fenwick2D {
    ll n = 0, m = 0;
    vector<vector<T>> bit;

    Fenwick2D() = default;
    Fenwick2D(int n_, int m_) {
        init(n_, m_);
    }

    void init(int n_, int m_) {
        n = n_;
        m = m_;
        bit.assign((int)n + 1, vector<T>((int)m + 1, T{}));
    }

    // add delta at (x, y), 0-indexed
    void add(ll x, ll y, T delta) {
        for (ll i = x + 1; i <= n; i += i & -i)
            for (ll j = y + 1; j <= m; j += j & -j)
                bit[i][j] += delta;
    }

    // sum over [0..x] x [0..y], 0-indexed; returns 0 if x<0 or y<0
    T sum_prefix(ll x, ll y) const {
        if (x < 0 || y < 0)
            return T{};
        T res{};
        for (ll i = x + 1; i > 0; i -= i & -i)
            for (ll j = y + 1; j > 0; j -= j & -j)
                res += bit[i][j];
        return res;
    }

    // sum over rectangle [x1..x2] x [y1..y2], inclusive, 0-indexed
    T sum_rect(ll x1, ll y1, ll x2, ll y2) const {
        return sum_prefix(x2, y2) - sum_prefix(x1 - 1, y2) - sum_prefix(
    x2, y1 - 1) + sum_prefix(x1 - 1, y1 - 1);
    }
};
```

3.4 Fenwick Tree

```
struct FenwickTree {
    vector<ll> tree;
    ll n;

    FenwickTree(ll N) {
        n = N;
        tree.assign(n, 0);
    }

    ll F(ll x) {
        return x & (x + 1);
    }

    ll H(ll x) {
        return x | (x + 1);
    }

    // [0, r)
    ll query(ll r) {
        if (r > n) {
            r = n;
        }
        --r;
        ll ans = 0;
        while (r >= 0) {
            ans ^= tree[r];
            r = F(r) - 1;
        }
        return ans;
    }

    // [l, r)
    ll query(ll l, ll r) {
        return query(r) ^ query(l);
    }
```

```
    void add(ll id, ll val) {
        while (id < n) {
            tree[id] ^= val;
            id = H(id);
        }
    }
};
```

3.5 HLD

```
struct HLD {
    ll n;
    vector<ll> top, pos, h, par;

    ll gpos = 0;

    SegmentTree st;

    void calc_sizes(ll v, vector<char>& used, const vector<vector<ll>>& g
    , vector<ll>& s) {
        s[v] = 1;
        used[v] = 1;

        for (ll u : g[v]) {
            if (!used[u]) {
                calc_sizes(u, used, g, s);
                s[v] += s[u];
            }
        }
    }

    void build_dec(ll v, ll p, ll pr, vector<char>& used, const vector<
    vector<ll>>& g, vector<ll>& s) {
        used[v] = 1;
        pos[v] = gpos++;
        top[pos[v]] = pos[p];
        par[pos[v]] = pos[pr];
        if (pr != v)
            h[pos[v]] = h[pos[pr]] + 1;

        vector<ll> child;

        for (ll u : g[v]) {
            if (!used[u]) {
                child.push_back(u);
            }
        }

        for (int i = 1; i < child.size(); ++i) {
            if (s[child[i]] > s[child[0]]) {
                swap(child[i], child[0]);
            }
        }

        if (!child.empty()) {
            build_dec(child[0], p, v, used, g, s);
            for (int i = 1; i < child.size(); ++i) {
                build_dec(child[i], child[i], v, used, g, s);
            }
        }
    }

    HLD(ll N, const vector<vector<ll>>& g, const vector<ll>& a) {
        n = N;

        top.resize(n, 0);
        pos.resize(n, 0);
        h.resize(n, 0);
        par.resize(n, 0);
        vector<char> used(n, 0);
        vector<ll> s(n, 0);

        calc_sizes(0, used, g, s);
        used.assign(n, 0);
        build_dec(0, 0, 0, used, g, s);

        vector<ll> pa(n, 0);
        for (int i = 0; i < n; ++i) {
            pa[pos[i]] = a[i];
        }

        st = SegmentTree(pa);
    }

    ll query(ll u, ll v) {
        ll ans = 0;

        u = pos[u];
```

```
        v = pos[v];

        ll pu = top[u];
        ll pv = top[v];

        while (pu != pv) {
            if (h[pu] >= h[pv]) {
                ans += st.query(pu, u + 1).sum;
                u = par[pu];
                pu = top[u];
            } else {
                ans += st.query(pv, v + 1).sum;
                v = par[pv];
                pv = top[v];
            }
        }
        ans += st.query(min(u, v), max(u, v) + 1).sum;

        return ans;
    }

    void change(ll u, ll val) {
        u = pos[u];
        st.change(u, val);
    }
};
```

3.6 Ordered Set

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

template <class T, class Compare = std::less<T>>
using ordered_set = tree<
    T,
    null_type,
    Compare,
    rb_tree_tag,
    tree_order_statistics_node_update>;
```

3.7 Persistent DSU

```
struct PersistentDSU {
    ll n;
    vector<ll> prev, size;

    PersistentDSU(ll N) : n(N) {
        prev.assign(n, -1);
        size.assign(n, 1);
    }

    ll root(ll a) {
        while (prev[a] != -1)
            a = prev[a];
        return a;
    }

    bool connected(ll a, ll b) {
        return root(a) == root(b);
    }

    vector<pair<ll, ll>> changes;  //

    bool unite(ll a, ll b) {
        a = root(a);
        b = root(b);
        if (a == b)
            return false;
        if (size[a] < size[b])
            swap(a, b);
        prev[b] = a;
        size[a] += size[b];
        changes.push_back({b, a});
        return true;
    }

    bool rollback() {
        if (changes.empty())
            return false;
        pair<ll, ll> ba = changes.back();
        changes.pop_back();
        prev[ba.first] = -1;
        size[ba.second] -= size[ba.first];
        return true;
    }
```

```
};
```

3.8 Prefix Automate

```
struct PrefixAutomate {
    const static int A = 26;

    struct Node {
        int next[A] = {-1};
        int link = -1;

        int prev = -1;
        int pe = -1;

        int has = 0;
    };

    vector<Node> tree;

    PrefixAutomate() {
        tree.push_back(Node());
    }

    void add(const string& s) {
        int v = 0;
        for (char c : s) {
            int e = c - 'a';
            if (tree[v].next[e] == -1) {
                tree[v].next[e] = tree.size();
                tree.push_back(Node());
                tree.back().prev = v;
                tree.back().pe = e;
            }
            v = tree[v].next[e];
        }
        tree[v].has = 1;
    }

    void addVect(const vector<string>& vs) {
        for (const auto& s : vs) {
            add(s);
        }
    }

    void del(const string& s) {
        int v = 0;
        for (char c : s) {
            int e = c - 'a';
            if (tree[v].next[e] == -1) {
                return;
            }
            v = tree[v].next[e];
        }
        tree[v].has = 0;
    }

    void ahoCorasick() {
        queue<int> que;

        tree[0].prev = 0;
        tree[0].pe = -1;

        tree[0].link = 0;
        for (int i = 0; i < A; ++i) {
            if (tree[0].next[i] == -1) {
                tree[0].next[i] = 0;
            } else {
                que.push(tree[0].next[i]);
            }
        }

        while (!que.empty()) {
            int v = que.front();
            que.pop();

            if (tree[v].link == -1) {
                if (v == 0 || tree[v].prev == 0)
                    tree[v].link = 0;
                else
                    tree[v].link = tree[tree[v].prev].next[tree[v].pe];
            }

            for (int i = 0; i < A; ++i) {
                if (tree[v].next[i] == -1) {
                    tree[v].next[i] = tree[tree[v].link].next[i];
                } else {
                    que.push(tree[v].next[i]);
                }
```

```
            }
        }
    }
};
```

3.9 Segment Tree Mass Simple

```
struct SegmentTreeMassSimple {
    struct Node {
        ll size, val, inc;

        Node(ll Val = 0, ll Size = 0, ll Inc = 0) {
            val = Val;
            size = Size;
            inc = Inc;
        }
    };

    ll n;
    vector<Node> tree;

    Node merge(Node n1, Node n2) {
        return Node(n1.val + n2.val, n1.size + n2.size);
    }

    SegmentTreeMassSimple(const vector<ll>& start) {
        n = start.size();

        tree.resize(n * 2);

        for (int i = 0; i < n; ++i) {
            tree[i + n] = Node(start[i], 1, 0);
        }

        for (int i = n - 1; i >= 0; --i) {
            tree[i] = merge(tree[i << 1], tree[i << 1 | 1]);
        }
    }

    void apply(ll i, ll val) {
        tree[i].val += val * tree[i].size;
        tree[i].inc += val;
    }

    void push(ll i) {
        if (i > 1) {
            push(i >> 1);
        }
        if (i < n) {
            apply(i << 1, tree[i].inc);
            apply(i << 1 | 1, tree[i].inc);
            tree[i].inc = 0;
        }
    }

    // fix node i and all parents
    void fix(ll i) {
        push(i);
        while (i > 1) {
            ll j = i >> 1;
            tree[j] = merge(tree[j << 1], tree[j << 1 | 1]);
            i = j;
        }
    }

    Node query(ll l, ll r) {
        l += n;
        r += n;

        Node ansl, ansr;

        push(l);
        push(r - 1);

        while (r > l) {
            if (l & 1) {
                ansl = merge(ansl, tree[l++]);
            }
            if (r & 1) {
                ansr = merge(tree[--r], ansr);
            }

            l >>= 1;
            r >>= 1;
        }

        return merge(ansl, ansr);
    }
```

```
    void change(ll l, ll r, ll inc) {
        l += n;
        r += n;

        ll l0 = l, r0 = r;

        push(l);
        push(r - 1);

        while (r > l) {
            if (l & 1) {
                apply(l++, inc);
            }
            if (r & 1) {
                apply(--r, inc);
            }

            l >>= 1;
            r >>= 1;
        }

        fix(l0);
        fix(r0 - 1);
    }
};
```

3.10 Segment Tree Mass

```
struct SegmentTreeMass {
    struct Node {
        ll sum = 0;
        ll add = 0;

        Node(ll Sum = 0, ll Add = 0) {
            sum = Sum;
            add = Add;
        }
    };

    Node neutral = Node();
    ll n;
    vector<Node> tree;

    SegmentTreeMass(const vector<ll>& start) {
        n = start.size();
        tree.resize(4 * n + 3);
        init(start);
    }

    SegmentTreeMass(ll N) {
        n = N;
        tree.resize(4 * n + 3);
        vector<ll> start(n, 0);
        init(start);
    }

    Node merge(Node n1, Node n2) {
        return Node(n1.sum + n2.sum);
    }

    void fix(ll v, ll l, ll r) {
        tree[v] = merge(tree[v * 2 + 1], tree[v * 2 + 2]);
    }

    void apply(ll v, ll l, ll r, ll val) {
        tree[v].add += val;
        tree[v].sum += val * (r - l);
    }

    void push(ll v, ll l, ll r) {
        ll m = (r + l) / 2;

        apply(v * 2 + 1, l, m, tree[v].add);
        apply(v * 2 + 2, m, r, tree[v].add);
        tree[v].add = 0;
    }

    void init(ll v, ll l, ll r, const vector<ll>& start) {
        if (l + 1 == r) {
            tree[v] = Node(start[l], 0);
            return;
        }

        ll m = (r + l) / 2;
        init(v * 2 + 1, l, m, start);
        init(v * 2 + 2, m, r, start);
        fix(v, l, r);
```

```
}

void init(const vector<ll>& start) {
    init(0, 0, n, start);
}

// [l: r)
Node query(ll v, ll l, ll r, ll ql, ll qr) {
    if (ql <= l && r <= qr) {
        return tree[v];
    }
    if (r <= ql || qr <= l) {
        return neutral;
    }

    ll m = (r + l) / 2;
    push(v, l, r);
    return merge(query(v * 2 + 1, l, m, ql, qr), query(v * 2 + 2, m,
r, ql, qr));
}

Node query(ll ql, ll qr) {
    return query(0, 0, n, ql, qr);
}

void add(ll v, ll l, ll r, ll ql, ll qr, ll val) {
    if (ql <= l && r <= qr) {
        apply(v, l, r, val);
        return;
    }
    if (r <= ql || qr <= l) {
        return;
    }

    ll m = (r + l) / 2;
    push(v, l, r);
    add(v * 2 + 1, l, m, ql, qr, val);
    add(v * 2 + 2, m, r, ql, qr, val);
    fix(v, l, r);
}

void add(ll ql, ll qr, ll val) {
    add(0, 0, n, ql, qr, val);
}
};
```

3.11 Segment Tree Simple

```
struct SegmentTreeSimple {
    ll n;
    vector<ll> tree;

    SegmentTreeSimple(ll N) {
        n = N;
        tree.resize(n * 2);
    }

    SegmentTreeSimple(const vector<ll>& start) {
        n = start.size();
        tree.resize(n * 2);

        for (int i = 0; i < n; ++i) {
            tree[i + n] = start[i];
        }
        for (int i = n - 1; i > 0; --i) {
            tree[i] = tree[i << 1] + tree[i << 1 | 1];
        }
    }

    void change(ll i, ll val) {
        i += n;
        tree[i] = val;
        while (i > 1) {
            tree[i >> 1] = tree[i] + tree[i ^ 1];
            i >>= 1;
        }
    }

    ll query(ll l, ll r) {
        ll ans = 0;

        l += n;
        r += n;

        while (l < r) {
            if (l & 1) {
                ans += tree[l++];
            }
```

```
            if (r & 1) {
                ans += tree[--r];
            }

            l >>= 1;
            r >>= 1;
        }

        return ans;
    }
};
```

3.12 Segment Tree

```
struct SegmentTree {
    struct Node {
        ll sum;

        Node(ll Sum = 0) : sum(Sum) {
        }
    };

    Node neutral = Node();

    ll n;
    vector<Node> tree;

    void init(ll v, ll l, ll r, const vector<ll>& start) {
        if (l + 1 == r) {
            tree[v] = Node(start[l]);
            return;
        }

        ll m = (l + r) / 2;

        init(v * 2 + 1, l, m, start);
        init(v * 2 + 2, m, r, start);
        tree[v] = merge(tree[v * 2 + 1], tree[v * 2 + 2]);
    }

    SegmentTree() {
    }

    SegmentTree(const vector<ll>& start) {
        n = start.size();
        tree.resize(n * 4 + 3);
        init(0, 0, n, start);
    }

    Node merge(Node n1, Node n2) {
        return Node(n1.sum + n2.sum);
    }

    Node query(ll v, ll l, ll r, ll ql, ll qr) {
        if (ql <= l && r <= qr)
            return tree[v];
        if (qr <= l || r <= ql)
            return neutral;

        ll m = (l + r) / 2;
        return merge(query(v * 2 + 1, l, m, ql, qr), query(v * 2 + 2, m,
r, ql, qr));
    }

    Node query(ll ql, ll qr) {
        return query(0, 0, n, ql, qr);
    }

    void change(ll v, ll l, ll r, ll id, ll val) {
        if (l + 1 == r && l == id) {
            tree[v] = Node(val);
            return;
        }
        if (r <= id || id < l) {
            return;
        }

        ll m = (l + r) / 2;
        change(v * 2 + 1, l, m, id, val);
        change(v * 2 + 2, m, r, id, val);
        tree[v] = merge(tree[v * 2 + 1], tree[v * 2 + 2]);
    }

    void change(ll id, ll val) {
        change(0, 0, n, id, val);
    }
};
```

3.13 Sparse Table

```
template <class T>
struct SparseTable {
    ll n, k;
    vector<T> a;
    vector<vector<T>> sparse;
    vector<ll> length;

    SparseTable(vector<T>& start) {
        n = start.size();
        k = 0;
        for (ll d = 1; d <= n; d *= 2, ++k) {
        }
        a.assign(start.begin(), start.end());
        sparse.assign(n, vector<T>(k));
        countSparse();
        precalc();
    }

    void precalc() {
        length.assign(n + 1, 0);
        ll t = 0;
        for (int l = 1; l <= n; ++l) {
            if ((1ll << (t + 1)) < l)
                ++t;
            length[l] = t;
        }
    }

    void countSparse() {
        for (int i = 0; i < n; ++i) {
            sparse[i][0] = a[i];
        }
        for (int j = 1; j < k; ++j) {
            for (int i = 0; i < n; ++i) {
                ll d = (1ll << (j - 1));
                sparse[i][j] = max(sparse[i][j - 1], sparse[min(n - 1, i
    + d)][j - 1]);
            }
        }
    }

    T query(ll l, ll r) {
        ll t = length[r - l];

        ll d = (1ll << t);

        return max(sparse[l][t], sparse[r - d][t]);
    }
};
```

3.14 Treap

```
struct Treap {
    Treap* left;
    Treap* right;
    ll x;
    unsigned int y;
    unsigned int size;
    ll dp;

    Treap(ll val = 0) {
        left = nullptr;
        right = nullptr;
        size = 1;
        x = val;
        y = mersenne();
        update(this);
    }

    friend ll get_dp(Treap* node) {
        if (node == nullptr)
            return 0;
        return node->dp;
    }

    friend unsigned int get_size(Treap* node) {
        if (node == nullptr)
            return 0;
        return node->size;
    }

    friend void update(Treap* node) {
        if (node == nullptr)
            return;
        node->size = 1 + get_size(node->left) + get_size(node->right);
```

```
        node->dp = max({node->x, get_dp(node->left), get_dp(node->right)
    });
    }

    // split by size
    friend pair<Treap*, Treap*> split_k(Treap* node, unsigned int k) {
        if (node == nullptr)
            return {nullptr, nullptr};
        if (get_size(node->left) < k) {
            auto res = split_k(node->right, k - get_size(node->left) - 1)
    ;
            node->right = res.first;
            update(node);
            return {node, res.second};
        } else {
            auto res = split_k(node->left, k);
            node->left = res.second;
            update(node);
            return {res.first, node};
        }
    }

    // split by dp: dp in left < k
    friend pair<Treap*, Treap*> split_dp(Treap* node, unsigned int k) {
        if (node == nullptr)
            return {nullptr, nullptr};
        if (get_dp(node) < k) {
            auto res = split_dp(node->right, k);
            node->right = res.first;
            update(node);
            return {node, res.second};
        } else {
            auto res = split_dp(node->left, k);
            node->left = res.second;
            update(node);
            return {res.first, node};
        }
    }

    friend Treap* merge(Treap* left, Treap* right) {
        if (left == nullptr)
            return right;
        if (right == nullptr)
            return left;
        if (left->y > right->y) {
            left->right = merge(left->right, right);
            update(left);
            return left;
        } else {
            right->left = merge(left, right->left);
            update(right);
            return right;
        }
    }

    friend Treap* insert(Treap* node, unsigned int pos, ll val) {
        auto res = split_k(node, pos);
        Treap* tr = new Treap(val);
        return merge(merge(res.first, tr), res.second);
    }

    friend Treap* erase(Treap* node, unsigned int pos) {
        auto res = split_k(node, pos + 1);
        auto left_res = split_k(res.first, pos);
        return merge(left_res.first, res.second);
    }

    // return {element, root}
    friend pair<Treap*, Treap*> kth_element(Treap* node, unsigned int k)
    {
        if (get_size(node) < k)
            return {nullptr, node};
        auto res = split_k(node, k + 1);
        auto left_res = split_k(res.first, k);
        return {left_res.second, merge(left_res.first, merge(left_res.
    second, res.second))};
    }

    // return sub segment [l, r) and remaineng Treap [0, l) + [r + 1,
    size)
    friend pair<Treap*, Treap*> cut(Treap* node, unsigned int l, unsigned
     int r) {
        auto res = split_k(node, r);
        auto left_res = split_k(res.first, l);
        return {left_res.second, merge(left_res.first, res.second)};
    }
};
```

4 Flows

4.1 Dinic

```
struct FlowSolver {
    struct DinicEdge {
        ll a, b, cap, flow;
    };

    vector<DinicEdge> edges;
    vector<vector<ll>> g;
    ll n;

    FlowSolver(ll N) {
        n = N;
        g.resize(n);
    }

    void add_edge(ll u, ll v, ll w) {
        g[u].push_back(edges.size());
        edges.push_back({u, v, w, 0});
        g[v].push_back(edges.size());
        edges.push_back({v, u, 0, 0});
    }

    bool bfs(ll s, ll t, vector<ll>& d) {
        queue<ll> q;
        d.assign(n, -1);
        d[s] = 0;
        q.push(s);

        while (!q.empty()) {
            ll v = q.front();
            q.pop();

            for (ll id : g[v]) {
                ll u = edges[id].b;
                if (d[u] == -1) {
                    if (edges[id].cap > edges[id].flow) {
                        d[u] = d[v] + 1;
                        q.push(u);
                    }
                }
            }
        }

        return d[t] != -1;
    }

    ll dfs(ll v, ll t, ll flow, vector<ll>& d, vector<ll>& ptr) {
        if (flow == 0) {
            return 0;
        }
        if (v == t) {
            return flow;
        }

        for (; ptr[v] < g[v].size(); ++ptr[v]) {
            ll id = g[v][ptr[v]];
            ll u = edges[id].b;

            if (d[u] == d[v] + 1) {
                ll add_flow = dfs(u, t, min(flow, edges[id].cap - edges[
id].flow), d, ptr);
                if (add_flow > 0) {
                    edges[id].flow += add_flow;
                    edges[id ^ 1].flow -= add_flow;
                    return add_flow;
                }
            }
        }
        return 0;
    }

    ll dinic(ll s, ll t) {
        ll flow = 0;
        vector<ll> d;
        while (bfs(s, t, d)) {
            vector<ll> ptr(n, 0);
            while (ll add_flow = dfs(s, t, inf, d, ptr)) {
                flow += add_flow;
            }
        }
        return flow;
    }
};
```

5 Graphs

5.1 2 SAT

```
struct SATSolver {
    ll n;
    vector<vector<ll>> g, gt;
    vector<char> used;
    vector<ll> order, comp;

    void init(ll N) {
        n = N;
        g.assign((int)(2 * n), {});
        gt.assign((int)(2 * n), {});
    }

    // literal: var in [0..n), val=false -> 2*var, val=true -> 2*var+1
    ll lit(ll var, ll val) { return 2 * var + (val ? 1 : 0); }
    ll neg(ll x) { return x ^ 1; }

    void add_imp(ll a, ll b) {
        g[(int)a].push_back(b);
        gt[(int)b].push_back(a);
    }

    // (a_var is a_val) OR (b_var is b_val)
    void add_or(ll a_var, ll a_val, ll b_var, ll b_val) {
        ll a = lit(a_var, a_val);
        ll b = lit(b_var, b_val);
        add_imp(neg(a), b);
        add_imp(neg(b), a);
    }

    void dfs1(ll v) {
        used[(int)v] = 1;
        for (ll to : g[(int)v]) if (!used[(int)to]) dfs1(to);
        order.push_back(v);
    }

    void dfs2(ll v, ll cl) {
        comp[(int)v] = cl;
        for (ll to : gt[(int)v]) if (comp[(int)to] == -1) dfs2(to, cl);
    }

    // returns empty vector if unsat; otherwise assignment[i] in {0,1}
    vector<ll> solve() {
        used.assign((int)(2 * n), 0);
        order.clear();
        for (ll i = 0; i < 2 * n; ++i) if (!used[(int)i]) dfs1(i);

        comp.assign((int)(2 * n), -1);
        ll j = 0;
        for (ll i = 2 * n - 1; i >= 0; --i) {
            ll v = order[(int)i];
            if (comp[(int)v] == -1) dfs2(v, j++);
        }

        vector<ll> ans((int)n);
        for (ll i = 0; i < n; ++i) {
            if (comp[(int)(2 * i)] == comp[(int)(2 * i + 1)]) return {};
            ans[(int)i] = (comp[(int)(2 * i)] < comp[(int)(2 * i + 1)]);
        }
        return ans;
    }
};
```

5.2 Euler Path

```
void cycle(ll prev, ll v, vector<vector<pair<ll, ll>>>& g, vector<ll>&
    ans) {
    while (!g[v].empty()) {
        auto iu = g[v].back();
        ll u = iu.second;
        g[v].pop_back();
        cycle(iu.first, u, g, ans);
    }
    if (prev >= 0)
        ans.push_back(prev);
}
```

5.3 Hungarian

```
// a size is (n + 1) x (m + 1), 1-indexing
// n <= m; if n < m than a[i][j] must be >= 0
pair<ll, vector<int>> hungarian(const vector<vector<ll>>& a, int n, int m
```

```
) {
vector<ll> u(n + 1, 0), v(m + 1, 0);
vector<int> p(m + 1, 0), way(m + 1, 0);
for (int i = 1; i <= n; ++i) {
    int j0 = 0;
    p[j0] = i;

    vector<char> used(m + 1, 0);
    vector<ll> minv(m + 1, inf);
    do {
        used[j0] = true;
        int i0 = p[j0];
        int j1 = -1, delta = inf;

        // run step of dfs from i0
        for (int j = 1; j <= m; ++j) {
            if (used[j]) {
                continue;
            }

            int cur = a[i0][j] - u[i0] - v[j];
            if (cur < minv[j]) {
                minv[j] = cur;
                way[j] = j0;
            }
            if (minv[j] < delta) {
                delta = minv[j];
                j1 = j;
            }
        }

        // update potentials
        for (int j = 0; j <= m; ++j) {
            if (used[j]) {
                u[p[j]] += delta;
                v[j] -= delta;
            } else {
                minv[j] -= delta;
            }
        }

        j0 = j1;
    } while (p[j0] != 0);

        while (j0) {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        }
    }

    return {-v[0], p};
}
```

5.4 Initial Pairing

```
int init_pairing(int n1, int n2, const vector<vector<int>>& g, vector<int
    >& mt, vector<int>& rmt) {
    auto PACK = [](int x, int y) { return ((ll)x << 30) + y; };
    auto SUNPACK = [](int x) {
        const static ll mask = (1ll << 30) - 1;
        return x & mask;
    };

    vector<int> deg(n1, 0);
    vector<vector<int>> ng(n1);
    vector<vector<pair<int, int>>> rg(n2);
    vector<int> next(n1, 0);
    set<ll> que;
    for (int i = 0; i < n1; ++i) {
        for (auto j : g[i]) {
            ++deg[i];
            rg[j].push_back({i, ng[i].size()});
            ng[i].push_back(j);
        }
        que.insert(PACK(deg[i], i));
    }

    int pairing = 0;

    while (!que.empty()) {
        auto res = *que.begin();
        que.erase(res);

        int i = SUNPACK(res);
        if (deg[i] == 0) {
            continue;
        }
```

```
        while (ng[i][next[i]] == -1) {
            ++next[i];
        }
        int j = ng[i][next[i]++];

        deg[i] = 0;
        for (auto ks : rg[j]) {
            int k = ks.first;
            int sz = ks.second;
            if (deg[k] != 0) {
                ng[k][sz] = -1;
                que.erase(PACK(deg[k], k));
                --deg[k];
                que.insert(PACK(deg[k], k));
            }
        }
        mt[j] = i;
        rmt[i] = j;
        ++pairing;
    }
    return pairing;
}

vector<int> fast_kuhn(int n1, int n2, vector<vector<int>>& g) {
    vector<char> used(n1, 0);
    vector<int> mt(n2, -1), rmt(n1, -1);

    init_pairing(n1, n2, g, mt, rmt);

    int mark = 0;

    for (int run = 1; run;) {
        ++mark;
        run = 0;
        for (int i = 0; i < n1; ++i) {
            if (rmt[i] == -1 && kuhn_dfs(i, used, mark, g, mt)) {
                run = 1;
            }
        }

        for (int j = 0; j < n2; ++j) {
            if (mt[j] != -1) {
                rmt[mt[j]] = j;
            }
        }
    }

    return mt;
}
```

5.5 Kuhn

```
bool kuhn_dfs(int v, vector<char>& used, char mark, vector<vector<int>>&
    g, vector<int>& mt) {
    if (used[v] == mark) {
        return false;
    }
    used[v] = mark;

    for (int u : g[v]) {
        if (mt[u] == -1 || kuhn_dfs(mt[u], used, mark, g, mt)) {
            mt[u] = v;
            return true;
        }
    }
    return false;
}

vector<int> kuhn(int n1, int n2, vector<vector<int>>& g) {
    vector<char> used(n1, 0);
    vector<int> mt(n2, -1);

    int mark = 1;

    for (int i = 0; i < n1; ++i) {
        if (kuhn_dfs(i, used, mark, g, mt)) {
            ++mark;
        }
    }

    return mt;
}
```

6 Number Theory

6.1 Eratosthenes

```
void erat(ll N, vector<ll>& primes, vector<ll>& mi) {
    mi.assign(N, 0);
    for (int i = 2; i < N; ++i) {
        if (mi[i] == 0) {
            primes.push_back(i);
            mi[i] = i;
        }

        for (int j = 0; j < primes.size() && primes[j] <= mi[i] && i *
    primes[j] < N; ++j) {
            mi[primes[j] * i] = primes[j];
        }
    }
}
```

6.2 Miller Rabin

```
bool _miller_rabin_test(ll n, ll d, ll s, ll a) {
    ll x = bin_pow_mod(a, d, n);
    for (int i = 0; i < s; ++i) {
        ll y = ((__int128_t)x * x) % n;
        if (y == 1 && x != 1) {
            return false;
        }

        x = y;
    }
    if (x != 1) {
        return false;
    }
    return true;
}

bool is_prime_ml(ll n) {
    if (n <= 1) {
        return false;
    }
    // all primes
    vector<ll> as = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    ll d = n - 1;
    ll s = 0;
    while ((d & 1) == 0) {
        ++s;
        d >>= 1;
    }
    for (ll a : as) {
        if (n == a) {
            return true;
        }
        if (n % a == 0) {
            return false;
        }
        if (!_miller_rabin_test(n, d, s, a)) {
            return false;
        }
    }
    return true;
}
```

7 Strings

7.1 Prefix Function

```
vector<ll> prefix_function(string s) {
    ll n = s.size();
    vector<ll> p(n, 0);
    for (int i = 1; i < n; ++i) {
        ll k = p[i - 1];
        while (k > 0 && s[i] != s[k])
            k = p[k - 1];
        if (s[i] == s[k])
            ++k;
        p[i] = k;
    }
    return p;
}
```

7.2 Z Function

```
vector<ll> z_function(string s) {
```

```
    ll n = s.size();
    vector<ll> z(n, 0);
    ll r = 0;
    for (int i = 1; i < n; ++i) {
        if (z[i - r] < r + z[r] - i)
            z[i] = z[i - r];
        else {
            z[i] = max(r + z[r] - i, 0ll);
            while (i + z[i] < n && s[i + z[i]] == s[z[i]])
                ++z[i];
            r = i;
        }
    }
    return z;
}
```